| Senior Director, Marketing Strategy & Platforms |
| Sales Dev Operations | Director, Sales Development Operations
| VP, Sales Development |
| Sales Compensation | Director, Sales Commissions
| Senior Director, Sales Operations |
| Legal | Director, Legal Compliance and Ethics
| VP, Legal |
| Sales Systems | Senior Manager, Sales Systems
| Senior Director, Enterprise Applications |
| Fulfillment | Group Manager, Fulfillment
| VP, Product Management |
| Data | Director, Data Analytics
| VP, Data & Analytics |

[Systems Owner] Systems Owner Sign-off

Salesforce CRM System Owners should provide the signoff. The signoff matrix is an below

| Main Approver | Backup Approver
|
|---|---
---|
| Sheela Viswanathan - Senior Manager, Sales Systems | Nabitha Rao - VP, IT
|
| Korben Carreno - Manager, CRM Systems | Raul Pavon - Director Enterprise
architecture and Applications |
| Kiran Chinthapalli - Director, CRM Systems | Nishanth Sekhar - Director,
Enterprise Applications (Lead to Cash) |
| | Monali Bhide - Manager, IT Enterprise
Applications Engineering |
| | Pratik Gupta - Manager, IT Enterprise
Applications Engineering |

[Systems DRI] Add the correct SalesSystems::Deployed - GitLab Label

Once the issue has been deployed, the issue should be tagged with one of the following deploy label following the SDLC - Software Development Life Cycle by the sales systems team member assigned to the issue

- SalesSystems::Deployed - 0 - No Changes
- SalesSystems::Deployed - 1 - Settings Change
- SalesSystems::Deployed - 2 - Configuration Change
- SalesSystems::Deployed - 3 - Code Change

[Systems DRI] Screenshot of Completed Change Set Attached and MR Attached (if Code)

If the issue ended up in label SalesSystems::Deployed - 2 - Configuration Change OR SalesSystems::Deployed - 3 - Code Change the systems member assigned to the issue should add the screenshot of the change set

Milestone Review and QA

Before a milestone can be closed, the following checks are performed by Sales Systems leadership:

1. All issues in the Sales Systems project and authored or assigned to a Sales Systems team member should have the Sales Systems Label.
1. All closed issues with the Sales Systems label should be assigned to a Milestone.
1. All closed issues in a Milestone need to make sure their SDLC steps below have been completed and have a final deploy label.

SFDC Development Guidelines

Before beginning work, make sure:

1. You have a fully setup local SFDC Dev Environment.
   - Visual Studio Code
   - Salesforce Trailhead: Setting up your VS Code
2. You have access to a personal SFDC Dev Sandbox.
3. Your SFDC Dev Environment is correctly pointed at your SFDC Dev Sandbox
4. You have cloned our Git repository into your local Sandbox working directory.
5. You are working from a GitLab issue with clear technical specifications that deliver on the agreed business requirements.
6. You have identified the priority of the request based on our priority matrix, and added the appropriate label: Priority::Low, Priority::Medium, Priority::High

Change Managment Steps:

1. Make sure you start on branch master and git pull.
2. Create a new branch, giving it a name that ties back to the issue: git checkout -b "SalesSystems-158".
3. If you are writing code, frequently push your changes to your sandbox using and SFDX: Deploy Source To Org on the changed classes, triggers or pages.
4. If you are editing configuration, frequently pull down your changes to your local environment using SFDX: Retrieve Source From Org on the changed objects or metadata.
5. Make sure to write and run a unit test for code, and for both code and config, test the changes by hand in the SFDC user interface.
6. When you feel your iteration is complete run git status to make sure the changed files are the ones you expected.
7. Add in the files you wish to commit with git add [filename] or git add if you want to add all changed files.
8. Commit your changes with a relevant message: git commit -m "Fixing Apex CPU Errors".
9. Using the link provided by GitLab, open a merge request, make it a Draft:, and assign it to the Architect on the project.
10. Comment on the related issue with an @ to the project's Architect for review, providing a link to the merge request. (this automatically links the merge request to the issue)
11. The Architect (or assigned delegate) will assign the story a Change Management level, based on the scope of the change as defined here.
12. You will then need to document that the appropriate approvals (as defined in the Approval Matrix section below) have been completed in the issue.

13. If the Architect calls for a live demo, schedule the meeting and prep your sandbox to do a run through with the end customer.
14. If the Architect calls for user acceptance testing, make sure the assigned testers have access to the sandbox where the work was done, and schedule testing.
15. Once the solution passes, the Architect will remove the WIP: status and merge the change.
16. Once merged, package up all relevant files into a Change Set from your Sandbox to Production (or to a Staging instance if the Architect requests it).
17. Name the Change Set the same as the issue/branch: SalesSystems-158 and push to production.
18. Once the Change Set arrives in production, validate it. If there are any errors, go back to step 3. If steps 3, 4, and 5 are followed errors at validation should be rare.
19. Once the Change set validates, ping the Architect to schedule the deployment.
20. After the deployment, perform any post deployment steps such as adding visibility to net new fields.
21. Confirm with the end user that the functionality is working as expected.
22. Create a merge request to our technical documentation adding the new feature or editing the features entry.
23. Before moving to your next task rebase with git checkout master then git pull. Always be pulling!
24. Clone the merged change set that was deployed into production and push and deploy this change set to staging. (Post deploy steps and setup are optionable)

Installed Packages

Installed packages are provided by ISVs (Independent Software Vendor) who work on the Salesforce platform, and contain the code and configuration for Salesforce which extend the capability of the platform. These are commonly installed and requested by our business partners to extend Salesforce's native capabilities.

Is the package Managed vs Unmanaged?

Packages come in two flavors, managed or unmanaged. Managed packages are equivalent to signed apps, with self contained source sealed inside of the package. Unmanaged packages are unsigned, and generally contain raw code and configuration.

Generally, vendors either provide managed packages via the Salesforce AppExchange, or via direct installation from their repository. Unmanaged packages generally are provided by the vendor, and may contain raw source or configuration which needs to be manually installed. Note, any code provided the GitLab remains the IP of the vendor provided, unless specific accommodations are provided (such as if we contract with a vendor to extend their base functionality). Because of these additional considerations, unmanaged packages require additional steps to be completed as part of the installation process.

Any package code is the responsibility of the vendor who produced it to support and troubleshoot. If issues are encountered with the functionality, please contact the vendor in question to troubleshoot. If there are changes recommended by the vendor to our environment, log an issue with Systems to track any changes which are requested by the vendor using one of the two processes below.

System stability comes first

We (Sales Systems) reserve the right to remove or uninstall managed or unmanaged code at any time, if this package is determined to cause issues related to system performance or limitations.

If we are accepting unmanaged code or config, we will choose whether to accept these on a case-by-case basis. Unmanaged packages offer significant risk and resource utilization over our managed code. Our goal is always to accept managed packages from vendors.

Managed Package Installation/Upgrade Process

1. Identify the package and what reason(s) you may think it should be installed or upgraded.
1. Install the version of the package you want to install inside of your sandbox environment.
1. Test and confirm the functionality provided meets your requirements, and has no negative impacts to existing functionality.
1. Open an issue with Sales Systems to install the package in the STAGING environment.
    - Ensure you include links to the package and the install instructions provided by the vendor in the description of the issue.
1. Once the package is installed in STAGING, if confirmed to move forward, test and confirm the functionality provided meets your requirements, and has no negative impacts to existing functionality
1. If successfully installed in STAGING, announce the intent to move forward in installing in production, and prepare training documentation.
1. Document any relevant information about the package as part of the handbook.
   - An example of this could be SFDC fields that are part of the package, and the business processes it supports.
1. Install the package in production, update the issue and close out.

Unmanaged Package Installation/Upgrade Process

1. Identify the package and what reason(s) you may think it should be installed or upgraded, along with any custom code or configuration which needs to be installed separately.
1. Install the version of the package you want to install inside of your sandbox environment.
1. Test and confirm the functionality provided meets your requirements, and has no negative impacts to existing functionality.
1. Open an issue with Sales Systems to install the package in the STAGING environment.
    - Ensure you include links to the package and the install instructions provided by the vendor in the description of the issue, along with the inventory of any components installed outside of the package.
1. Create a branch in our Git repository to include any custom code or metadata created as part of the unmanaged package which will be tracked in GitLab source.
1. Request a package review of the branch by Systems, by assigning the MR to the Sales Systems developers.
1. Once the package is installed in STAGING, if confirmed to move forward, test and confirm the functionality provided meets your requirements, and has no negative impacts to existing functionality
1. If successfully installed in STAGING, announce the intent to move forward in installing in production, and prepare training documentation.
1. Document any relevant information about the package as part of the handbook.
   - An example of this could be SFDC fields that are part of the package, and the business processes it supports.

1. Install the package in production, update the issue and close out.

Installed Package Removal Process

The uninstall process is the same regardless of whether a package is managed or unmanaged.

1. Identify the package and what reason(s) you may think it can be removed.
2. Perform initial research on what the packages original intent may have been and identify who owns/owned the use of the functionality.
   - GitLab's Tech Stack Google Sheet is a great place to check for this information and can be found here
3. Open an issue with the owner to investigate further. In this discussion, obtain confirmation on whether or not it may be removed.
   - Add the technical debt label to the issue.
4. If confirmed to move forward, test by removing the package from the sandbox.
5. If successfully removed from sandbox, announce the intent to move forward in removing.
6. Document any relevant information about the package.
   - An example of this could be SFDC fields that are part of the package.
7. Remove the package from production, update the issue and close out.

Field & Process Deprecation

- Since field & process deprecation is as common an occurrence as the creation it is important that the system team implements a repeatable process that we can leverage when deprecating any fields pr processes.

Field Deprecation

- This process is most often used by the systems team. If you have or are aware of a field in Salesforce that is no longer needed, please inform the Sales Systems team by following the process outlined in getting help from the sales systems team

1. Open an issue listing out all of the fields that we are investigating to deprecate. Be sure to include the field name, field API name and the object that the field is associated with in a table in the description of the issue. Add the technical debt label to the issue.
2. Alert the data team to the upcoming field deprecation by tagging them on the issue.
3. Alert all relevant partner teams (Marketing Ops, Sales Ops, Finance Ops etc.) as needed
4. Prepend [DEPRECATE] to the beginning of the field name. If the field name cannot accommodate a field name that long copy and paste the original name into the description, trim unnecessary characters from the name and try again. For this reason [DELETE] is also acceptable to prepend to the field name.
5. In Visual Studio Code, pull from master and perform a scan for each of the API names in the issue. If the field is used, investigate if the code can be updated as to not include this field.
6. If code is updated in the previous step prepare a merge request and relate it to the issue.
7. If your sandbox is out of date, work with the Systems team to refresh it so that any recent edits are included in the next step.
8. Push any updated code to your sandbox (if applicable) and start a change set.
9. For all fields that are still eligible to be deprecated log into your sandbox and attempt to delete them one by one. Record any connection between any fields and any field updates,

workflow rules, validation rules etc. (Reports, Report Types etc can be ignored in this step)
10. Investigate any connections found in the previous steps and if the field can still be deleted.
11. For all fields that cannot be deleted

- Link the investigation issue to the investigated field by pasting the GitLab Issue Link in the fields description.
- Assign someone as an owner of the field in Salesforce

12. For all fields that can be deleted

- List them out on a final comment on the issue
- Update the due date of the issue to the date they will be deleted
- Confirm that there are no issues with the tagged related teams
- Validate any change sets with updated automations (if applicable) before the issue due date
- On the issue due date deploy any change sets and delete the fields from production. If possible allow for a 1 day lag time between field deletion and deleting fields from the Deleted Fields section in Salesforce

Process Deprecation

- Deprecating a process often includes a change in team behavior as well as updates to any processes. The Systems team is working on detailed documentation to address these changes and more info will be coming soon!

Deactivate Service User

- This deactivation process is made to deactivate service user profiles. Service accounts are accounts that are used as integration Users, Connection users etc., in order to deactivate the service user account follow the template. Please note deactivating standard users will be done by Sales Operations.

Sales System's journey with CI/CD using GitLab and Salesforce

We have begun the journey of further leveraging our own GitLab tool by creating our first pipeline for our own Salesforce environment!

Our own pipeline is based on the great work done by @mayanktahil and @francispotter: the SFDC CI/CD templates. If you are interested in more information about this project and want to see it in action, check out Salesforce Development with GitLab and Accelerate DevOps with GitLab and Salesforce

With this comes some change, as we are now more stricly enforcing compliance controls by limiting manual changes into the STAGING org.

Effective 2/16/2022, the following methods are the only approved way to deploy to STAGING.

- Via inbound change sets from another GitLab owned sandbox
- Via vendor package installs or upgrades

- Automatic deployments via our CI/CD pipeline

Our CI/CD template

Our own version of this CI/CD template can be found here. It is a simplified version, to allow us to iterate.

This template removes the capabilities to use Scratch Orgs in favor of using an Org-based deployment model. The model used by this CI/CD file has a single environment configured, denoted as 'STAGING'. The deployment script also limits deployments only to ApexClasses, ApexTriggers, ApexPage, and ApexComponents stored in the root source directory.

The pipeline performs the following action:

- Whenever a commit is made to a branch related to a Merge Request, install the SFDX CLI on a runner and execute the sfdx force:source:deploy method to perform a validation deployment against STAGING.
- This validation deployment will compile all ApexClass, ApexTrigger, ApexComponent, and ApexPage objects found in the new commit source branch.
- If the compile succeeds, all unit tests in the SANDBOX org will be executed to confirm all unit tests are passing.
- If the compile or unit tests fails, the pipeline will spit out the errors as individual line items in the output of the job.
  - The MR will then be blocked from merging.
- If the compile succeeds and unit tests pass, the MR will be cleared for merging after code review is complete.
  - The pipeline will also allow for the user to trigger a deployment of the source code to the STAGING environment.
  - This is a manual process for now (see What's next?) and will be triggered by the person who is merging the MR once the merge has completed.
    - The team decided to leave this step manual so that we have flexibility on deployments in case multiple MRs were being merged simultaneously.
    - In this scenario, we will only deploy the last MR as it will have the final complete 'master' branch will all previous MRs merged.

Benefits of the pipeline

- All channges to ApexClasses, ApexTriggers, ApexPage, and ApexComponents stored in the Sales Systems source will now be managed directly from source, rather than having to be managed manually through change sets or via manual deploys.
- We reduce the number of manual changes to the STAGING environment, limiting potential conflicts and issues creeping into production.
- We can leverage the power of GitLab Analytics to better understand how we can better run our team!

What's next?

We are beginning to explore using Sandbox Source Tracking, a feature Salesforce released last year to enable easy export of configuration changes from a developer environment into source control.

This tool will enable our admins to track complex changes to their developer orgs and easily check these into source control.

Once we do so, we can expand our pipeline to include these objects in our pipeline in STAGING. This will allow us to validate administrative changes such as field renames, picklist value changes, validation rules, workflow, or flows, and deploy them quickly to STAGING. As this removes the manual step for admins to build change sets from their environments into STAGING, it will save them time to focus on other things.

After this, our next goal will be to see if we want to start automating deployments to STAGING once an MR is merged. This will only save us a click, but is an important step for us as a team to become comfortable with using the process of automated deployments into our STAGING environment.

SECTION: sales/field-operations/sales-systems/dataloader-installation.md

Installing from Scratch

1. Go to Salesforce Dataloader's GitHub repo's Releases
1. Please download one major version previous from the latest, but at minimum version 56.0.6. The file should be named something like dataloadermacv whatever version you want to download .zip
   - EXAMPLE: If the latest release version is 30.9.2, you'll want to download version 29.x.y, where x and y are the highest subversions of major version 29
   - WHY? Salesforce regularly updates its API but delays the new version from production environments
      - If you download the latest release version and try to use it in production, but production is not updated to that API version yet, your upload probably won't work
      - Newest API versions are released as a preview in sandbox environments, so the latest version of the dataloader should always work in sandboxes, but you may have to wait till it works in prod
1. Remember if your Mac is using an Apple processor. If you're not sure, follow these substeps
   - From any window, top left corner of the screen should be an Apple symbol, click that
   - Click About this Mac
   - Look for Processor or Chip
      - If you see "Intel" anywhere on that line, you <ins>do not</ins> have an Apple processor
      - If you see "Apple M"something on that line, you <ins>do</ins> have an Apple processor
1. If you do not have Java installed, or you're not sure if Java is installed, follow these substeps. If you're really sure you already have Java installed, go to the next numbered Step.
   - If you <ins>do not</ins> have a Mac processor
      - Install Java from GitLab's Self Service app
         - From any window, top right of the screen should be a magnifying glass
         - Click that and search for Self Service
            - If you see Self Service.app show up, open that
            - If you don't see Self Service.app anywhere, ask IT about it
         - Open Self Service app and search Java
         - Install "Java"version number"- Open Source"

- Really unlikely, but if the version number is < 11, please let someone on the Sales Systems team know, Dataloader requires at least Java version 11 to run
- If you <ins>do</ins> have an Apple processor
- We'll be downloading Java directly, you need a slightly different version than what the Self Service app provides
- Go to Azul JDKs and scroll down till you see a button to download a .dmg file
- The link in the above substep should already have a few options filled in for you, Java 18, macOS, ARM 64-bit, and JDK. If these aren't set, or this version of Java is no longer available, or you also want/need to use Salesforce CLI, please let someone on Sales Systems know
- Click the .dmg button and download it
- When it's done downloading, open the .dmg file and install Java. If it asks you about where to install it, the defaults should be OK
- Open a terminal window and paste:
- /usr/sbin/softwareupdate --install-rosetta
- Hit "enter"
- This command is necessary to have Dataloader run on these new processors as explained in the "Running Data Loader on Mac M1 Hardware" section of the official Salesforce Dataloader install instructions
- Don't follow Salesforce's instructions, this handbook page will explain the rest of the steps you need

1. Find the zip file for Dataloader you downloaded from Step 2 and extract it
1. From the extracted folder, hold down the "control" key and click the "install.command" file, from the dropdown, click "Open"
1. If a window pops up, click "Open" again
1. A terminal window should pop up. This is the Dataloader install wizard
1. First thing it should be asking is where to install. The default path is fine, just hit "enter"
1. It should then ask if you want a desktop shortcut. I suggest you type Yes and hit "enter" to create a shortcut
1. The last thing it should ask is if you want to add an Applications shortcut. Type Yes if you do or No if you don't then hit "enter". We usually do Yes
1. The installation should be finished! You can close the terminal window if it looks like there were no problems
- Ask someone on Sales Systems if it looks like there's a problem in the terminal window

1. You can delete the zip file and the extracted folder
1. Try opening Dataloader from one of the shortcuts if you made one. Tell someone on Sales Systems if it doesn't open or if there are any errors while opening.

Uninstalling Dataloader

1. If you added a shortcut to Applications for the version of Dataloader you want removed
- Go to Applications in Finder and delete the shortcut

1. If you added a shortcut to Desktop for the version of Dataloader you want removed
- Go to Desktop in Finder and delete the shortcut

1. If you know where you installed the version of Dataloader you want removed
- Go there and delete the folder
- Skip the rest of the numbered steps, you're done!

1. Try using Search to find where it was installed
- From any window, top right of the screen should be a magnifying glass, click that

- Type dataloader
- There should be a folder in the results, click that folder to bring up a Finder window
- In that folder, if there's only one sub folder with a version number, you can delete the whole "dataloader" folder
- If there's multiple versions, delete the versions you don't want and keep the ones you want to keep. To delete all of them, delete the whole "dataloader" folder

1. If you can't find where it was installed, don't worry too much, you should be able to have different versions installed at the same time with no conflicts, or ask someone on the Sales Systems team for help

Upgrading Versions

1. Follow the Uninstalling Dataloader instructions
   - Note the version(s) of Dataloader you're uninstalling
   - If at least one of them is version 45 or higher and you know it worked,
     - You most likely have Java installed, when you are following the "Installing from Scratch" steps, you probably don't have to install Java
   - If none of the versions are at least 45
     - You may need to install Java, follow those steps in the "Installing from Scratch" section
1. Follow the Installing from Scratch instructions

SECTION: sales/field-operations/sales-systems/gtm-integrated-environments.md

How to use this documentation

This page provides a guide to show how our main GTM SaaS applications and their sandboxes are connected and what the purpose of each integrated environment layer is.

Production

Support	E-Commerce	Billing	Sales	Marketing
Zendesk Production	customers.gitlab.com Production	Zuora Production	Salesforce Production	Marketo Production

Our production systems are integrated by either stock or internally developed integrations (and see image below) and serve as the template for our other environments to emulate.

Status: Integrated and Running.

Staging

Support	E-Commerce	Billing	Sales	Marketing
Zendesk "Sandbox"	customers.gitlab.com Staging	Zuora "Central" Sandbox	Salesforce "Staging" Sandbox	Marketo "Sandbox"

Our Staging environment is designed to emulate production as closely as possible. Changes from the Development environment should be deployed here first for user acceptence testing before being promoted to production.

Status: Currently we are using the Interim setup for both staging and development.

Development

Support	E-Commerce	Billing	Sales	Marketing
N/A	customers.gitlab.com Engineer Dev Environment	Zuora "apiSandbox2" Sandbox	Salesforce "Sandbox" Sandbox	N/A

Our integrated development environment is designed for working on projects delivering large functionality changes between our systems. The current use for this setup will be our Zuora conversion to Orders and the Zuora 360 and Zuora CPQ upgrades associated with that.

Status: Currently we are using the Interim setup for both staging and development.

Interim

Support	E-Commerce	Billing	Sales	Marketing
N/A	customers.gitlab.com Staging	Zuora "apiSandbox1" Sandbox	Salesforce "Sandbox" Sandbox	N/A

This is our current development environment being used to work on Web Directs send more data to Salesforce. It will be depreciated and repurposed once that work is done.

Other Development Environments

The Sales Systems team uses the "1 Sandbox Per Engineer" rule for ease and speed of development, this might cause for deployments directly into Staging or doing a full stop off in the Development Environment.

SECTION: sales/field-operations/sales-systems/salesforce-config.md

Salesforce Config

The purpose of this page is to document configuration of our SFDC org. This will serve as the "go-to" place to check in regards to questions on our general configuration.

Salesforce Provisioning

For roles that should automatically receive Salesforce access your account and permissions will be automatically created by Okta. For anyone else who needs Salesforce access for their job responsibilities, please open an Access Request.

Salesforce De-Provisioning

- User Off-Boarding - Users will be automatically deactivated by OKTA based on off-boarding issues. If OKTA is not able to deactivate users, a systems issue will be created to unblock the dependency and system team members will manually deactivate the users.
- Salesforce Licence Harvesting - User Management Team will run a monthly user report for last login > 60 Days. Based on the inactivity, users list will be provided to OKTA Team to de-activating users. Related AR issue will be created and the deactivated users will be communicated through Salesforce Licence Harvesting Slack Channel.
- Service Account(s) - Service Account user will be manually de-activated by Systems Team and OKTA will not be able to de-provision integration user(s).

Salesforce Installed Packages

This has moved to the Internal Handbook

SFDC Certificates And Updating Expiring Certificates

Salesforce Certificates

Learn more about Salesforce Certificates and Keys here

Updating Expiring Certificates

The Salesforce knowledge base has a resource that addressed what to do and how to handle Expiring certificates. Currently in Salesforce our certificate is located in two places and needs to be updated in both. In order to update please follow the below steps and update in the following locations.

- Create a new certificate by searching for Certificate and Key Management in setup. From there create a self-signed certificate and ensure that the options match the certificat you are replaceing. Please note that the information in the Certificate field will be slightly differnt between the old an new certificate. Then update the certificate in the following locations
  - SAML Single Sign-On Settings
    - To update the certificate here update the certificate in the Request Signing Certificate picklist. (Do not upload the Salesforce Created Certificate into the file Identity Provider Certificate)
  - Identity Provider
    - To update the certificate here update the certificate in the Label picklist

Critical SFDC Permissions

Critical Permission	Systems Administrator	Sales Operations	All Other Profiles	Permission Set Assigned to Individuals
Deploy Change Sets	Yes	No	No	No
Customize Application	Yes	No	No	No
Manage Users	Yes	Yes	No	Yes - Sales Comp Team

SFDC Backups

Our Salesforce Backup solution is Ownbackup. There compliance information is located here.

SECTION: sales/field-operations/sales-systems/super-sonics.md

Related Handbook Sections

- Internal Team Handbook

Dynamic Quote Templates Automation

The Dynamic Quote Templates Automation, also known as Super Sonics, is a controlled flow that combines user input, automatically detected qualities of the quote and historical selections in order to automatically produce quote templates with the desired legal language on them as well as syncing with Zuora in order to control the actual behavior of the system. There are a number of features that can either be selected or are automatically added. For more information on these features please see our internal handbook here - (link coming soon).

FAQ

- How do I add language to the quote?
  - There are specific fields on the quote layout, that either begin with [Language] or [Cloud Lic] that a user can manually select to have language added to the quote

- I clicked one of the [Language]/[Cloud Lic] checkboxes and I can't produce the quote
  - It is possible that you have selected a combination of quote language and line items that are not permitted. In this case you may either receive an error or you may see that approvals are required for your quote.
  - If you receive an error please review the error language and attempt to resolve by deselecting the mentioned checkboxes as needed.
  - If approvals are required you can review the approvals that are needed in the quote approvals section.

- I need language added to the template that the automation will not permit me to add - how do

I get it added?
  - In the event that you need language added to the quote that the system is not set up to support please reach out to Sales Support for manual assistance. They are able to review and approve these requests as well as assist in overriding the system to produce the required quotes.

- There is language appearing on the quote that I didn't select - how/why?
  - There are a number of scenarios that SFDC automatically scans for on a quote. For example we scan for MSA's on Quotes. If it's detected that an MSA is associated with a quote we will automatically put MSA language onto the quote.
  - If you believe this automatically added language was added to your quote incorrectly please reach out to Sales Support for additional help

- How do I request help if I still have questions?
  - If you still have questions or are encountering errors you can reach out to Sales Support for additional help

 Logic Locations

- ZuoraQuoteClass
  - quoteTemplatePlinkoBoard
- ZuoraQuoteTest
  - quoteTemplatePlinkoBoard
  - quoteTemplatePlinkoBoardHighlightedSkus
  - quoteTemplatePlinkoBoardExistingSub
  - quoteTemplatePlinkoBoardExistingMSA
  - quoteTemplatePlinkoBoardGCP
- ZuoraQuoteTrigger

SECTION: sales/field-operations/sales-systems/tech-stack-guide-sfdc.md

Salesforce Tech Stack Guide

> Note: Refer to the Tech Stack Index to browse Apps and Tech Stack Applications to manage Apps.

{{% tech-stack "Salesforce" %}}

 Implementation

Salesforce consists of several modules and installed packages.

 System Diagrams

TBD

 Data Model

TBD

Integrations

Zuora to Salesforce

Zuora Data to Salesforce via Zuora CPQ

Go-To-Market Production SaaS Environments

Go-To-Market Integrated Environments

Key Reports / Dashboards

TBD

SECTION: sales/field-operations/sales-systems/gtm-technical-documentation/_index.md

How to use this documentation

The documentation below is organized by feature. Each section will have links to the appropriate business process page, the relevant inputs and outputs for the feature, as well as references to the specific logic that processes the input and outputs.

ARR

Please see the dedicated ARR Technical Documentation Page

Gainsight

Please see the dedicated Gainsight Technical Documentation Page

Xactly

More information to come. If you need a new field brought into Xactly please leverage the AddFieldToXactlySales issue template in the Sales Systems Project

Territory Success Planning

Business Process this supports: Territory Success Planning

Overview: The goal of TSP is to keep a set of staging fields consistently up to date from a variety of data sources, then at given intervals copy these values to the "Actual" set of fields for general use. This allows for us to constantly receive changes but only apply those

changes in a controlled fashion. This also allows us to easily track exceptions. Note: This project was originally referred to as ATAM, which is why the API names of the fields reference that instead of TSP.

Logic Locations: AccountJob.cls

Code Units:

- highestEmpsAndTSPAddress
- ownerTransfer

Inputs: DataFox, Zoominfo, Manually Entered Address & Employee Data, Account Parenting Hierarchy

Outputs: Here is the outline between two sets of fields we are setting on the Account object. The Staging(TSP / ATAM) fields are set nightly via an APEX job. Actuals are set at given intervals found in the business documentation.

Data Name	Actual - Field API Name	TSP - Field API Name
Owner	Owner	ATAMApprovedNextOwnerc
Owner Role	Owner.Role	ATAMNextOwnerRolec
Owner Team	AccountOwnerTeamc	ATAMNextOwnerTeamc
Sales Segment	UltimateParentSalesSegmentEmployeesc	JBTestSalesSegmentc
Territory	AccountTerritoryc	ATAMTerritoryc

Landed Addressable Market (LAM)

Business Process this supports: Landed Addressable Market (LAM)

Overview: LAM is a measure of how much more expansion or growth we can achieve with the customers we already have. This value is calculated in Salesforce daily based on several inputs.

Logic Locations: AccountJob.cls, AccountJobSetParentLAMFields.cls

Code Units:

- AccountJobSetZuoraSubInfo (all methods)
- AccountJobSetParentLAMFields (all methods)
- AccountTrigger
- AccountClass.SetLAMOnAccounts

Inputs: Zuora, Zoominfo,LinkedIn, Manually Entered Employee Data, Account Parenting Hierarchy,

Outputs: Here are the various fields used in the solution, along with how they are set.

The LAM calculation runs in four parts:

1. The AccountJob executes and calculates subscription data for all Accounts.

1. The AccountJobSetZuoraSubInfo executes and calculates subscription data for each Account.
1. The AccountJobSetParentLAMValues executes and calculates subscription data across Account hierarchies.
1. The AccountTrigger fires when the Account's LAM calculation changes, and sets the 'LAMc' field based on logic for each territory and geo.

Data Name	Actual - Field API Name	Data Type	Set By
LAM	LAMc	Currency	AccountTrigger, AccountClass.SetLAMOnAccounts
LAM Tier (Industry)	LAMTierIndustryc	Formula	SFDC
LAM Tier (Dev Count)	LAMTierc	Formula	SFDC
LAM Seat (Industry)	LAMSeatIndustryc	Formula	SFDC
LAM Seat (Dev Count)	LAMSeatc	Formula	SFDC
LAM Price (Industry)	LAMPriceIndustryc	Formula	SFDC
LAM Price (Dev Count)	LAMPricec	Formula	SFDC
LAM Max (Industry)	LAMMaxIndustryc	Formula	SFDC
LAM Max (Dev Count)	LAMMaxc	Formula	SFDC
LAM Dev Count	LAMDevCountc	Formula	SFDC
LAM: Ultimate Annualized Seat Price	LAMUltimateAnnualizedSeatPricec	Formula	SFDC
LAM:Ult. Avg Seat Price, this Account	LAMUltAvgSeatPricethisAccountc	Formula	SFDC
LAM:Starter Avg Seat Price, this Account	LAMStarterAvgSeatPricethisAccountc	Formula	SFDC
LAM: Starter Annualized Seat Price	LAMStarterAnnualizedSeatPricec	Formula	SFDC
LAM: Prevalent Tier Avg Price, this Acct	LAMPrevalentTierAvgPricethisAcctc	Formula	SFDC
LAM: Prevalent Tier Avg Price, Acct Fam	LAMPrevalentTierAvgPriceAcctFamc	Formula	SFDC
LAM:Premium Avg Seat Price, this Account	LAMPremiumAvgSeatPricethisAccountc	Formula	SFDC
LAM: Premium Annualized Seat Price	LAMPremiumAnnualizedSeatPricec	Formula	SFDC
LAM: Est Dev Percent by Industry	LAMEstDevPercentbyIndustryc	Formula	SFDC
LAM: Count of Ultimate Subscriptions	LAMCountofUltimateSubscriptionsc	Currency	AccountJobSetZuoraSubInfo
LAM: Count of Ultimate Subs, Acct Family	LAMCountofUltimateSubsAcctFamilyc	Currency	AccountJobSetParentLAMFields
LAM: Count of Starter Subscriptions	LAMCountofStarterSubscriptionsc	Currency	AccountJobSetZuoraSubInfo
LAM: Count of Starter Subs, Acct Family	LAMCountofStarterSubsAcctFamilyc	Currency	AccountJobSetParentLAMFields
LAM: Count of Premium Subscriptions	LAMCountofPremiumSubscriptionsc	Currency	AccountJobSetZuoraSubInfo
LAM: Count of Premium Subs, Acct Family	LAMCountofPremiumSubsAcctFamilyc	Currency	AccountJobSetParentLAMFields
LAM:Aggregate Annual Ultimate Seat Price	LAMAggregateAnnualUltimateSeatPricec	Currency	AccountJobSetParentLAMFields
LAM: Aggregate Annual Starter Seat Price	LAMAggregateAnnualStarterSeatPricec	Currency	AccountJobSetParentLAMFields
LAM: Aggregate Annual Premium Seat Price	LAMAggregateAnnualPremiumSeatPricec	Currency	AccountJobSetParentLAMFields
LAM: Acct Below Land Line	LAMAcctBelowCutLineFormc	Formula	SFDC

CARR (Acct Family)	CARRAcctFamilyc	Formula	SFDC
CMRR (Acct Family)	CMRRAllChildAccountsc	Currency	AccountJobSetParentLAMFields
Number of Licenses Sold (This Account)	NumberofLicensesThisAccountc	Number	AccountJobSetZuoraSubInfo
Ultimate license count	UltimateLicenseCountc	Number	AccountJobSetZuoraSubInfo
Starter license count	StarterLicenseCountc	Number	AccountJobSetZuoraSubInfo
Prevalent Tier (Account)	PrevalentTierc	Formula	SFDC
Prevalent Tier (Hierarchy)	PrevalentTierHierarchyc	Formula	SFDC
Premium license count	PremiumLicenseCountc	Number	AccountJobSetZuoraSubInfo
Parent LAM: Aggregate Developer Count	AggregateDeveloperCountc	Formula	SFDC
Estimated Developer Count	EstimatedDeveloperCountc	Formula	SFDC
Parent LAM: Sum Ultimate Seat Price	ParentLAMSumUltimateSeatPricec	Currency	AccountJobSetParentLAMFields
Parent LAM: Sum Starter Seat Price	ParentLAMSumStarterSeatPricec	Currency	AccountJobSetParentLAMFields
Parent LAM: Sum Premium Seat Price	ParentLAMSumPremiumSeatPricec	Currency	AccountJobSetParentLAMFields
Parent LAM: Sum of Num of Licenses	ParentLAMSumofNumofLicensesc	Number	AccountJobSetParentLAMFields
Parent LAM: Max ZI Number of Developers	ParentLAMMaxZINumberofDevelopersc	Number	AccountJobSetParentLAMFields
Parent LAM: Max Potential Users	ParentLAMMaxPotentialUsersc	Number	AccountJobSetParentLAMFields
Parent LAM: LinkedIn Developer Count	ParentLAMMaxDecisionMakersLinkedInc	Number	AccountJobSetParentLAMFields
Parent LAM: Industry (Acct Heirarchy)	ParentLAMIndustryAcctHeirarchyc	Picklist	AccountJobSetParentLAMFields
Parent LAM: Count of Ultimate Subs	ParentLAMCountofUltimateSubsc	Number	AccountJobSetParentLAMFields
Parent LAM: Count of Starter Subs	ParentLAMCountofStarterSubsc	Number	AccountJobSetParentLAMFields
Parent LAM: Count of Premium Subs	ParentLAMCountofPremiumSubsc	Number	AccountJobSetParentLAMFields

Contact Ownership

Business Process this supports: This supports our contact ownership rules

Overview: The goal of the Contact Ownership code is to ensure that contacts are owned by the appropriate user within salesforce in an automated fashion so that contact ownership is maintained without any work needed by team members.

Logic Locations: ContactJob
Code Units:

- maintainContactOwnership

Inputs: Contact Owner, Account Owner, Account SDR Assigned, Account Type, Sales Segment

Outputs: Contact Owner

Salesforce SAFE: Record Sharing And Visibility Settings

Please see our internal document for details.

For manual sharing of opportunity record, please refer Field Operations handbook here

Quote Approval System

Business Process this supports: Discount Approvals

Overview: According to the Deal Approval Matrix Quotes must have discounts approved by different management levels depending on discount percentage and term length. To achieve this, we have written automation to stamp a quote with each potential approver, revised the code that determines which approvals are required, and revised the actual approval process in Salesforce.

Quote Management Stamp When a Quote is inserted, get the owner of the related Opportunity. Then, find the manager of the owner and the manager of the manager for each manager, five managers down. Record the first active Regional Director, Area Sales Manager, and Vice President on the Quote. These lookup fields will be used in the Approval Process, if needed.

Quote Approval Code This is a table of the Quote (API Name: zquQuotec) fields that trigger quoteApprovals to recalculate and what must happen to them.

Field API Name	What Must Happen
RatePlanCountc	Change
zquPreviewedTCVc	Change
zquPreviewedSubTotalc	Change
zquPreviewedDiscountc	Change
NonStandardContractTermsc	Change
ResellerPOStatusc	Change
zquPaymentTermc	Change
zquPreviewedTotalc	Change
zquPreviewedDiscountc	Change
QuoteAmendmentLastModifiedDatec	Change
zquInitialTermc	Change
zquRenewalTermc	Change
XTriggerQuoteApprovalCheckc	Become true

If any of these events happen, all "RequiredApprovals" fields (RequiredApprovalsFromCEOc, RequiredApprovalsFromCFOc, RequiredApprovalsFromCROc, RequiredApprovalsFromCSc, RequiredApprovalsFromLegalc, RequiredApprovalsFromVPofChannelc, RequiredApprovalsFromVPofSalesRDc, RequiredApprovalsFromRDc, RequiredApprovalsFromASMc) are cleared. These are the rich text area fields that show which management levels need to approve the Quote on the page layouts.
Then, all relevant Quote Rate Plan Charges (API Name: zquQuoteRatePlanChargec) related to the

Quote are queried, these are what hold the term, product, and discount information we need to determine what approvals are required. Following the Deal Approval Matrix, we determine what level of management the Quote Rate Plan Charge needs and stamp the correct "RequiredApprovals" fields with the discount percentage, type, and term. Similar logic is then run for any Quote Rate Plan Charges related to Professional Services products.
Finally, the Quote's ApprovalStagec field records whether it needs approval, doesn't need approval, or has been approved.

Quote Approval Process
This utilizes Salesforce's built-in Approval Process functionality.
We have two Approval Processes for Zuora Quotes, the first for undiscounted, and the other for ones with discounts.
The Quote must be submitted using the "Submit for Approval" button on the page layout to enter the correct Approval Process.

- Undiscounted Approval Process
  - If the Quote's Approval Stage is "Approvals Not Required" or null, the Approval Stage is updated to "In Review" and the Owner of the Quote is emailed confirming submission for approval. Then, if there are any Special Terms and Notes or has been flagged as Requires Deal Desk Review, a member of the Deal Desk team must approve. If neither of those are true, the deal auto-approves. Upon approval, the Owner of the Quote is emailed to inform them of the approval and the Approval Stage is updated to "Approved". If the Quote is rejected, the Approval Stage is set to "Rejected" and the Owner is emailed informing them.
- Discounted Approval Process
  - If the Quote's Approval Stage is "Approvals Required" or "Rejected", the Approval Stage is updated to "In Review" and the Owner of the Quote is emailed confirming the submission for approval. Then, based on the "RequiredApprovals" fields, the Quote waits for approval by the people in that step. Once all approvals are acquired, the Approval Stage is set to "Approved" and the Owner of the Quote is emailed. If any approval step is rejected, the Approval Stage is set to "Rejected" and the Owner is emailed as well.

Logic Locations:

- ZuoraQuoteClass.cls
Code Unit:
  - stampManagerStack
- ZuoraQuoteClass.cls
Code Unit:
  - quoteApprovals
- Salesforce Approval Process Setup
Manage Approval Process For:
  - Quote (Installed Package: Zuora Quotes)

Salesforce Chatter to Cases

Business Process this supports: The field needs a streamlined process to address their concerns on specific salesforce records within salesforce. This is also used by the finance team to help address record specific billing issues, as well as the Community Advocate team to manage the influx of requests the team receives.

Overview: The goal of the Chatter To Cases functionality is to allow a streamlined communication channel that the field can leverage while also providing a streamlined case management system for the supporting team members to manage the requests that are sent to them from the field. If a team member uses an appropriate tag in salesforce a salesforce case record will automatically be created. Once these records are created supporting team members can work through the respective cases that are created to address the needs and concerns of the field team.

Inputs: Chatter text within Salesforce

Outputs: Salesforce Case Records

Logic Locations:
Code Units:

- Triggers
  - ChatterFeedCommentTrigger.trigger
  - ChatterFeedItemTrigger.trigger
- Clases
  - ChatterFeedCommentClass.cls
  - ChatterFeedItemClass.cls
- Tests
  - ChatterFeedItemTest.cls
  - ChatterFeedCommentTest.cls

Supported Groups

- @sales-support: Used by the Deal Desk team to manage inbound requests from the Sales Team.
- @billing ops: Used by the Billing team to manage inbound requests as they pertain to Billing.
- @revenue: Used by the Revenue team to review Opportunities and how we will record revenue.Detailed Response Here.
- @SMB Flat Renewals: Used by our SMB team for flat renewal support. Please see this Section of the handbook for how this is used.
- @Partner Help Desk: Used by the Channel Partner Help Desk (PHD) Team. Please see this Section of the handbook for more information.
- @Sales-Comp: Used by our Compensation Team and should be used to reach out to them regarding, splits, compensation etc. as it pertains to specific opportunities.
- @Partner-Ops: Used by our Partner Operations team and should be used to reach out to them - Link to their business section coming soon
- MktgOps-Support: Used by our Marketing Operations team and should be used to reach out to them - Link to their business section coming soon

Steps to add a Group:

- Do to limitations with Salesforce much of the minor updates must be implemented manually in production
- (In Production:Pre Deployment) Create a Chatter Group with the alias that you want the end users to be able to chatter in Salesforce.
- (In Production:Pre Deployment) Add a picklist value to the Origin field on the case object
- (Changeset) Update the ChatterFeedCommentClass and the ChatterFeedItemClass to monitor for

the use of the Chatter Group in chatters within Salesforce
- (Changeset) Update the CaseClass to include the new groups Id so that it updates the case owner what ownd by this queue.
- (Changeset) Create a Queue that will own the Case until it is automatically switched into a user's name who will work the case.
- (In Production: Post Deployment) Review Queue member and email options with requester

Related Epic

- @Sales-Ops Case Epic

Legal Request System

Business Process this supports The sales cycle, if a GitLab sales rep encounters an issue that requires legal knowledge, opinion, or action.

Overview A sales rep can quickly and easily create a Case for our legal team directly from an Opportunity's page layout in Salesforce. The legal team has access to a Salesforce dashboard to see how many Cases have been created for them, how many are in their name, etc. Clicking the "Legal Request" button on each Opportunity's page will bring the user to a page that asks a few questions that the legal team would like to know. Once the page is submitted, a Case is created with the Origin marked as "Legal Request." The legal team has dashboards that view Cases with Origin equal to "Legal Request" and can assign and take action from there.

Logic Locations

- Custom Buttons:
  - In Setup, under Opportunity, "Buttons, Links, and Actions", Legal Request
- Visualforce Pages:
  - LegalCaseCreate.page
- Apex Classes:
  - LegalCaseCreateController.cls

Primary Quote System

Business Process this supports The sales cycle and the financial processes around deals.

Overview We are now ensuring Opportunities in Salesforce have only one Quote that is marked as Primary. If multiple Quotes are being inserted under an Opportunity marked as primary in the same transaction, only the first in the list will be the primary. If a Quote is being inserted as primary, and there is an existing primary Quote, the existing will become not-primary and the incoming will be the new primary. If more than one Quote under the same Opportunity is being updated to become primary in the same transaction, an error message will prevent the update. A primary quote will not be allowed to be deleted. To change which Quote is primary, simply navigate to the Quote you want to be primary and update it as such, the previous primary Quote will automatically be updated to no longer be primary.

Logic Locations

- ZuoraQuoteClass.cls

Code Unit:
- primaryCheck

Opportunity Stage Progression Tracking

Business Process this supports The sales cycle and analytics.

Overview The goal of this functionality is to capture the progression of an opportunity through the stages in the sales process. To support this, a checkbox and date stamp will be automatically populated for each stage as the opportunity advances and or moves back in stage. The tracking begins at stage 0-Pending Acceptance. In the event an opportunity advances forward and skips stages, all of the skipped stages will be stamped with the same date as the resting stage. In the event of Closed Lost and Unqualified, the checks and dates will only apply for the stages that were actually met. To avoid data loss and confusion, the stage progression tracking will only run once, the first time through the stages.

Logic Locations

- To be added once functionality is in Production

Opportunity Validation Rule for Closed Accounting Period

Business Process this supports The Sales Finance & Analytics

Overview The goal is to have a static opportunity population in salesforce once the opportunity close date is past accounting close date (which is 10th day of the month) so it ties to everything that was reported to the Board. And to ensure the bookings data don't change as there are other downstream implications such as Commissions Calculation, Booking Reporting & Adaptive Bookings Data,need to build a mechanism to lock the booking related opportunity fields after green light from Deal Desk & Finance. If there are any minor adjustments (if needed) to historical periods we have specific permission sets to make the appropriate changes.

Logic Locations

- Validation Rule

Block Salesforce From Transferring Historical Opp Owners On Account Owner Transfers

Business Process this supports: In order to provide reliable and accurate historical data to the analytics team, the sales organization and to the company as a whole we need to ensure that historical opportunities and relevant information on opportunities is not changed once the opportunity is closed.

Overview: The goal of this blocking logic is to close a backdoor that Salesforce has built into the system. While we have a number of validation rules in place to prevent information from changing on closed opportunities it is possible to change historical opportunity owners (as well as fields that are derived from the owner field) while transferring accounts. Anyone who could have been able to change the owner on an account would have been able change historical opportunity data that they would not be able to edit otherwise. This logic still allows users to complete this account ownership transfer without any impact to historical opportunities

while also allowing the various business teams at GitLab to manually update the owners of opportunities at month close.

Inputs: Account records that are changing ownership

Outputs: Reverts opportunity owners to their original owners if the user attempted to change them

Logic Locations:

- Code Units:
  - ProtectClosedOppOwnersBefore
  - ProtectClosedOppOwnersAfter
- Triggers
  - AccountTrigger.trigger
- Classes
  - AccountClass.cls
- Tests
  - AccountClassTest.cls

Order Type System

Business Process this supports: New vs Connected New vs Growth

Overview: The goal of the Order Type system is to determine a given Opportunity's relationship with the business. Did it start a new customer relationship, cross into a related segment of the customer, or grow an existing relationship.

Design Presentation: https://docs.google.com/presentation/d/1G95aExDMTT1of6TkVWMPsx1FhNp3sNBl43
1t5PsKZmE/edit?usp=sharing

Logic Locations: AccountJob.cls

Code Units:

- determineOpportunityRevenueTypes

Inputs: Salesforce Account Hierarchy, Salesforce Opportunity Close Date and Stage.

Outputs: Populates the Order Type field on the Opportunity with New - First Order, New - Connected or Growth based on the following logic:

Order Type	Description
New - First Order	The First Closed Won Opportunity in an Account Family.
New - Connected	The First Closed Won Opportunity on an individual Account, that is not the first one in the Account Family.
Growth	All opportunities that follow the New - First Order or New - Connected opportunities. This includes Add-ons, Renewals, and additional Subscriptions.

Lead Segmentation

Business Process this supports: Sales Segmentation

Overview: Leads should be sorted into different Sales Segments based on their company's employee count so the appropriate salesperson can pursue them. We have a number of different information sources to get company size, so we must also establish a hierarchy for them.

Info Source	Salesforce Lead Field API Name
Lean Data	LeanDataMatchedAccountSalesSegmentc
Web Portal	WebPortalPurchaseCompanySizec
Marketing	EmployeeBucketsc
DemandBase	DBEmployeeCountc
Zoominfo	ZIEmployeesc
Salesforce User	NumberOfEmployees

Logic Locations: LeadClass.cls
Code Unit:

- determineSegment

Force Management / Command of The Message / Command Plan

Business Process this supports: Command of The Message

Overview: This Visualforce page and supporting controller provide the sales team with an easy to use button on their opportunities to populate the needed information.

Logic Locations:

- ForceManagement.cls
- ForceManagement.page

Linear Attribution

Please see our internal document for details.

Mavenlink

Business Process this supports: This supports our professional services team. They leverage Mavenlink projects to coordinate their projects, the hours they spend on each project and their associated tasks, schedules and more.

Overview: The following sections of code control the process by which Mavenlink projects in Salesforce are created, which in turn are then pushed over to Mavenlink by leveraging an extension class that was provided by Mavenlink. Currently a Mavenlink Project is created when one of the following scenarios is met

- A primary quote is created, that has a flagged Quote Rate Plan Charge on it (Mavenlink flag),

where its associated opportunity is in stage 5 or later and their is not already an existing Mavenlink project for the related Opportunity
- A Opportunity is moved into stage 5 or later, and it's primary quote has a flagged Quote Rate Plan Charge on it (Mavenlink flag) and their is not already an existing Mavenlink project for the related Opportunity
- In the above two cases if there is an associated Mavenlink project the project is updated with the new updated information that has been changed

Logic Locations:

- OpportunityClass.CreateAndMaintainMavenLinkProject
- QuoteRatePlanChargeClass.CreateAndMaintainMavenLinkProject
- MavenlinkProjectClass.upsertMavenLinkProject
- GitlabMavenlinkExtension.cls
- OpportunityClassTests.CreateAndMaintainMavenLinkProject
- QuoteRatePlanChargeClassTest.CreateAndMaintainMavenLinkProject
- GitlabMavenlinkExtensionTest.cls

Opportunity Splits

Business Process this supports: This supports the automatic creation validation of our opportunity split that supports our compensation team. This helps ensure that our team members are compensated for the opportunities that they are associated with in an automated fashion

Overview:

- Split Creation and Automation
- Below we'll see some key points as they pertain to Opportunity splits and below that we attempt to summarize the automation by end user story.
- Split for any Opportunity can only be created by an individual on one of the teams Below. To clarify the current permission does not aim to say who should be creating opportunity splits but rather who can create them based on our current permission set assignments.
    - Compensation Team
    - Deal Desk
    - Sales Ops
    - System Admins
- Account Executives / Opportunity Owner
  - All of these splits should only ever appear in Opportunity - Incremental ACV 2 split type
  - When the Opportunity Owner is updated, the splits for the Opportunity Owner are updated.
  - If a split is needed for the Owner the split needs to be created manually by an approved user
- Sales Development Representatives / Primary Solutions Architect / Channel Manager: Base split automation rules
  - When the corresponding lookup field on the Opportunity is populated (created or updated) a split for 100% is created for the user in the lookup field and added to the opportunity
  - The population of the above lookup fields follow the same rules and processes as they have before the rollout of this automation
  - If a lookup field is changed from User A to User B then ALL splits for that User Role on the Opportunity are deleted and a split for 100% is assigned to User B
  - If a lookup field is changed from a User to Null/Empty then ALL splits for that User Role

on the Opportunity are deleted, and there will be not splits for that Team Role on the Opportunity
  - If a split is needed for any of these roles the split needs to be created manually by an approved user
- Customer Success Manager Special Use Cases
  - This is handeled through the CSM Stamping process for some teams and are aligned with the SA Team for other CSM Teams. Splits aren't relevant for Customer Success Managers and Compensation. Please see CSM Team Stamping on this page for more details
- Channel Manager Special Use Cases
  - This is handeled through several matrixes that either stamp channel managers on the Opportunity and diffffernt opportunity splits depending on a nuber of layers of criteria.
    More detail coming soon to the handbook. Refernce this Epic and related Issues in the meantime

Split Validation

- OpportunityClass.checkAndConfirmSplitPercentages
  - When an Opportunity has its stage changed there is a validation run against the splits on the opportunity. The validation aims to ensure that all splits on the Opportunity when grouped by Role add up to 100%. If the splits do not add up to 100% an error is thrown and the splits mus